Università degli Studi di Catania

Dipartimento di Matematica e Informatica

Corso di Artificial and Swarm Intelligence
Relazione Progetto

Warehouse Swarm Intelligence System

Simulazione Multi-Agente per il Recupero di Oggetti
in Ambienti di Magazzino

Autore:
Giuseppe Bellamacina
Matricola: 1000030349

Professore del corso:
Mario Francesco Pavone

Anno Accademico 2025/2026

Indice

1 Struttura Comune: BaseAgent **3**
1.1 Percezione e Mappa Locale . . . 3
1.1.1 Vision Explored . . . 3
1.1.2 Tombstone per Oggetti Stale . . . 3
1.1.3 Redirect Relay → Dubious (solo Retriever) . . . 4
1.2 Integrazione della Mappa Condivisa . . . 4
1.3 Protocollo ClearWay . . . 4
1.3.1 Yield Cooldown . . . 4
1.4 Navigazione nei Magazzini: Direzionalità delle Porte . . . 4
1.5 Rilevamento dello Stallo . . . 5

2 Scout Agent **6**
2.1 Ciclo Decisionale . . . 6
2.1.1 Relay Passivo e Parametro seek_coordinator . . . 7
2.2 Selezione delle Frontiere . . . 7
2.2.1 Funzione di Utilità Completa . . . 7
2.2.2 Filtro Zone Esplorate (Solo Unknown Mode) . . . 8
2.2.3 Blocco del Target (Target Lock) . . . 8
2.2.4 Blacklist dei Target Recenti . . . 8
2.3 Routing a Zone (Map_Known) . . . 9
2.3.1 Suddivisione della Griglia . . . 9
2.3.2 Selezione della Zona Target . . . 9
2.3.3 Filtro Hard delle Frontiere . . . 9
2.4 Anti-Clustering . . . 10
2.5 Pattugliamento Ciclico (Stale Coverage) . . . 10

3 Coordinator Agent **11**
3.1 Raccolta delle Informazioni . . . 11
3.2 Logica di Assegnazione . . . 11
3.3 Strategia di Movimento . . . 12
3.3.1 Riposizionamento verso il Centroide . . . 12
3.3.2 Pattugliamento Anti-Inerzia (Boredom Patrol) . . . 12
3.3.3 Ricerca Attiva dei Retriever (Seek-Retriever) . . . 12
3.4 Consapevolezza Spaziale . . . 12
3.4.1 Rilevamento Chokepoint . . . 12
3.5 Esplorazione in Modalità map_known . . . 13
3.6 Sincronizzazione tra Coordinator Multipli . . . 13

4 Retriever Agent **14**
4.1 Gerarchia delle Priorità . . . 14
4.2 Auto-Assegnazione con Quattro Livelli di Sicurezza . . . 15
4.3 Cancellazione Reattiva dei Task . . . 15
4.4 Sequenza di Consegna al Magazzino . . . 15
4.5 Selezione del Magazzino Ottimale . . . 16
4.5.1 Ri-routing per Congestione . . . 16
4.5.2 Ordinamento delle Celle Interne (Queueing) . . . 16
4.6 Coordinamento tra Retriever (Peer Negotiation) . . . 17
4.6.1 Broadcast Peer-to-Peer degli Oggetti Scoperti . . . 17
4.6.2 Relay delle Posizioni . . . 17
4.7 Tolleranza ai Guasti: Cargo Drop . . . 17
4.8 Sistema degli Oggetti Dubbi . . . 17
4.8.1 Potatura con Confronto Timestamp . . . 18
4.9 Frontiere di Esplorazione . . . 18
4.9.1 Persistenza del Momento alla Frontiera . . . 18
4.9.2 Esplorazione in Modalità `map_known` . . . 19

5 Benchmarks **20**
5.1 Preset A . . . 20
5.1.1 Snapshot Finali — Preset A . . . 23
5.2 Preset B . . . 25
5.2.1 Snapshot Finali — Preset B . . . 28

Capitolo 1

Struttura Comune: BaseAgent

Tutti e tre i tipi di agente ereditano da `BaseAgent`, che implementa le funzionalità condivise: percezione dell'ambiente, ricezione e invio messaggi, movimento tramite A*, gestione dell'energia e rilevamento dello stato di stallo.

1.1 Percezione e Mappa Locale

Ogni agente mantiene una *mappa locale* rappresentata come matrice numpy, inizialmente tutta `UNKNOWN`. Ad ogni passo, la fase `STAGE_SENSE` aggiorna le celle entro il raggio di visione con il tipo reale della griglia, applicando la logica di *ray casting*: le celle dietro un ostacolo non vengono aggiornate anche se geometricamente entro il raggio, simulando la visibilità lineare degli agenti fisici. Gli oggetti nella zona di visione vengono aggiunti alla lista `known_objects`; le celle magazzino vengono aggiunte a `known_warehouses`.

1.1.1 Vision Explored

Oltre alla mappa locale, ogni agente mantiene una matrice separata `vision_explored` di dimensione `grid_height` $\times$ `grid_width`, inizializzata a zero. Una cella viene marcata a 1 esclusivamente quando entra nel cono di visione diretto dell'agente (ray casting), indipendentemente dalle informazioni ricevute via `MapDataMessage`. Questa distinzione è fondamentale nella modalità `map_known`: quando la topologia della griglia è pre-caricata, `local_map` non contiene celle `UNKNOWN` e la ricerca di frontiere classica fallisce. In questo caso tutti gli agenti usano `vision_explored == 0` come maschera di inesplorato, trattando le celle non ancora scansionate visivamente come frontiere da esplorare. Ciò garantisce che gli oggetti sparsi sulla griglia vengano effettivamente individuati, anche se la topologia è già nota.

1.1.2 Tombstone per Oggetti Stale

Quando un agente scansiona direttamente una cella che in precedenza conteneva un oggetto e la trova vuota (oggetto già raccolto), registra un *tombstone* in `known_objects_cleared` con il passo corrente. Qualsiasi messaggio ricevuto in seguito che affermi la presenza dell'oggetto in quella cella viene rifiutato se il suo timestamp è anteriore al tombstone. Questo impedisce la re-inserzione di oggetti fantasma nella mappa condivisa.

Oltre al tombstone passivo, il BaseAgent mantiene un insieme `_confirmed_gone`: ogni cella che l'agente ha scansionato e trovato vuota viene registrata in questo set. Quando deve costruire un `MapDataMessage`, l'agente esegue uno *scrubbing* preventivo: ogni cella con tipo `OBJECT` che è in `_confirmed_gone` oppure non corrisponde a un oggetto realmente presente

sulla griglia viene declassata a `FREE` nel messaggio e nella propria mappa locale. Questo interrompe la catena di ritrasmissione di informazioni obsolete attraverso lo sciame.

1.1.3 Redirect Relay → Dubious (solo Retriever)

Il `MapDataMessage` trasporta anche gli oggetti noti del mittente. Quando il BaseAgent li integra in `known_objects`, il Retriever, nella propria implementazione di `process_received_messages`, intercetta questa operazione: prima di invocare `super()`, registra in `_known_before` una copia di `known_objects`; subito dopo, sposta tutte le posizioni *nuove* (aggiunte dal relay ma non viste direttamente) in `dubious_objects`. In questo modo solo la visione diretta costituisce conoscenza confermata, mentre le informazioni di relay sono trattate come indizi da verificare (Sezione 4.8).

1.2 Integrazione della Mappa Condivisa

Alla ricezione di un `MapDataMessage`, l'agente aggiorna le celle della propria mappa ancora `UNKNOWN` con i dati del mittente. Il merge estrae anche le celle magazzino, popolando `known_warehouses`: in questo modo i Retriever scoprono le stazioni di ricarica ben prima di visitarle, purché uno Scout o un Coordinator vicino le abbia già esplorate.

1.3 Protocollo ClearWay

Quando un agente calcola un percorso A* verso il proprio obiettivo ma la cella successiva è occupata da un altro agente fermo, anziché attendere o ricalcolare un percorso alternativo, invia un `ClearWayMessage` direttamente all'occupante. L'agente ricevente, se libero da vincoli immediati, si sposta sulla cella adiacente più vicina libera. Questa negoziazione è ricorsiva (`depth` nel messaggio) fino a una profondità massima: se la catena di spostamenti non si risolve, l'agente iniziante rilascia il messaggio e lascia che A* trovi una via alternativa al passo successivo. Il meccanismo riduce significativamente i blocchi in corridoi stretti senza richiedere alcuna struttura di prenotazione globale.

1.3.1 Yield Cooldown

Dopo aver ceduto una cella in risposta a un `ClearWayMessage`, l'agente registra la posizione in `_yield_cooldown` con un timer di 3 passi. In questo periodo A* penalizza fortemente quella cella, impedendo il ritorno immediato e prevenendo oscillazioni con il richiedente.

1.4 Navigazione nei Magazzini: Direzionalità delle Porte

La navigazione A* applica un sistema di *forbidden types* contestuale per garantire la direzionalità delle porte del magazzino:

- Un agente **all'interno** del magazzino che si dirige verso l'uscita proibisce le celle `ENTRANCE` come nodi intermedi, impedendo che il percorso passi dalla porta di ingresso.
- Un agente **all'esterno** che si dirige verso l'ingresso proibisce le celle `EXIT` come nodi intermedi, impedendo scorciatoie attraverso l'uscita.

- Un agente in navigazione esterna generica proibisce **tutte** le celle porta (sia ingresso che uscita) come nodi intermedi, evitando che il percorso attraversi i magazzini come scorciatoia.

La cella obiettivo è sempre esclusa dal set di proibizioni, garantendo che l'agente possa effettivamente raggiungere un ingresso o un'uscita quando è la sua destinazione.

1.5 Rilevamento dello Stallo

L'agente registra lo storico delle posizioni recenti. Se per un certo numero di passi consecutivi (`stuck_threshold`) la posizione non cambia, il campo `stuck_counter` viene incrementato. Superata la soglia, l'agente esegue una *stuck recovery*: invalida il percorso corrente, aggiunge le posizioni degli agenti vicini come evitate e ricalcola un percorso alternativo verso una destinazione leggermente ruotata. Se lo stallo persiste, l'agente si sposta temporaneamente verso una cella casuale libera per sbloccarsi.

Quando il rilevatore di loop o di stallo abbandona un target, la posizione viene inserita nella blacklist `unreachable_targets` con il passo corrente: in questo modo il ciclo decisionale dell'agente non riassegna immediatamente lo stesso obiettivo irraggiungibile nel passo successivo.

Capitolo 2

Scout Agent

Lo Scout è ottimizzato per la copertura massima della griglia nel minor numero di passi. Non recupera oggetti e non riceve compiti: la sua unica responsabilità è esplorare e condividere informazioni.

2.1 Ciclo Decisionale

La priorità dello Scout è deterministica e segue questa sequenza:

1. **Ricarica energetica**: Se l'energia scende sotto la soglia critica (calcolata come la distanza stimata al magazzino più vicino moltiplicata per un fattore di sicurezza), lo Scout interrompe l'esplorazione e si dirige al magazzino. Durante la ricarica, continua a comunicare la mappa locale e gli oggetti scoperti.

2. **Consegna scoperte al Coordinator**: Se lo Scout ha scoperto nuovi oggetti e un Coordinator si trova nel raggio di comunicazione, li trasmette immediatamente tramite `ObjectSpottedMessage`. Se nessun agente è stato in contatto negli ultimi `_SEEK_COORDINATOR_DELAY` passi (default 25) e il parametro `seek_coordinator` è attivo, lo Scout si dirige verso l'ultima posizione nota del Coordinator per consegnare le informazioni. Se la posizione salvata risulta stale (il Coordinator non è presente all'arrivo), viene effettuata una scansione estesa fino a 4× il raggio di comunicazione per localizzarlo. La posizione del Coordinator viene aggiornata sia dalla visione diretta sia dalle posizioni ricevute in relay via `MapDataMessage`.

3. **Selezione frontiera**: Se non è in ricarica, `FrontierExplorer` calcola le frontiere disponibili dalla mappa locale e seleziona quella con utilità massima. In modalità `map_known`, il routing a zone (Sezione 2.3) filtra le frontiere per dirigere lo Scout verso la zona più inesplorata.

4. **Movimento**: A* calcola il percorso ottimale verso la frontiera selezionata, con le posizioni degli altri agenti come penalizzazione soft.

5. **Comunicazione**: Ad ogni passo, lo Scout condivide con tutti gli agenti entro il raggio `comm_radius` la propria mappa locale aggiornata tramite `MapDataMessage` e la lista degli oggetti scoperti tramite `ObjectSpottedMessage` indirizzati ai Coordinator vicini.

2.1.1 Relay Passivo e Parametro seek_coordinator

Il `MapDataMessage`, trasmesso ad ogni passo a tutti gli agenti nel raggio di comunicazione, trasporta anche la lista degli oggetti noti. Le scoperte si propagano quindi passivamente attraverso qualsiasi catena di agenti: scout → retriever → coordinator. Lo Scout attiva la ricerca attiva del Coordinator solo quando è stato *completamente isolato* (nessun contatto con alcun agente) per almeno `seek_coordinator_delay` passi.

Quando il parametro booleano `seek_coordinator` è disattivato, lo Scout non cerca mai attivamente il Coordinator: si affida esclusivamente al relay passivo. In alcune configurazioni di mappa, disabilitare questo parametro riduce i passi sprecati in spostamenti non esplorativi.

2.2 Selezione delle Frontiere

2.2.1 Funzione di Utilità Completa

La funzione di utilità per la selezione delle frontiere combina cinque termini:

$$U(f) = \frac{S_{\text{eff}}(f)}{d(f)^{0.4} + 1} - 2 \cdot P_{\text{locale}}(f) - P_{\text{globale}}(f) + B_{\text{cov}}(f) + B_{\text{mom}}(f)$$

`backend/algorithms/exploration.py | FrontierExplorer.select_best_frontier`

Dimensione Effettiva (Scaling Multiplicativo per Copertura). La dimensione del cluster viene scalata dalla frazione inesplorata della zona circostante:

$$S_{\text{eff}}(f) = \text{cluster_size}(f) \cdot \max\big(1 - r_{\text{exp}}(f),\ 0{,}05\big)$$

`backend/algorithms/exploration.py | FrontierExplorer.select_best_frontier`

dove $r_{\text{exp}}(f)$ è la frazione di celle già esplorate nella finestra W_k centrata sulla frontiera ($k = 11$ in modalità unknown, $k = 7$ in `map_known`):

$$r_{\text{exp}}(f) = \frac{|\{c \in W_k(f) : c \neq \texttt{UNKNOWN}\}|}{|W_k(f)|}$$

`backend/agents/scout_agent.py | ScoutAgent.step_decide`

In questo modo, una frontiera in una zona esplorata all'89% con cluster di dimensione 21 ha $S_{\text{eff}} = 21 \times 0{,}11 \approx 2{,}3$: non può competere con piccole frontiere in territorio completamente inesplorato.

Penalizzazione Locale.

$$P_{\text{locale}}(f) = \sum_{\substack{a \in \text{agenti_vicini} \\ d(f,a) < 10}} 0{,}5$$

`backend/algorithms/exploration.py | FrontierExplorer.select_best_frontier`

Ogni agente entro 10 celle dalla frontiera aggiunge una penalizzazione di 0,5. Il fattore è raddoppiato ($\times 2$) nella formula finale.

Penalizzazione Globale.

$$P_{\text{globale}}(f) = \sum_{t \in \text{peer_targets}} \max\Big(0,\ \frac{8 - d(f,t)}{8}\Big) \cdot 1{,}5$$

`backend/algorithms/exploration.py | FrontierExplorer.select_best_frontier`

Penalizzazione più debole basata sui target di esplorazione degli altri Scout, propagati via relay. Deconflitto a campo lontano.

Bonus Copertura.

$$B_{\text{cov}}(f) = (1 - r_{\text{exp}}(f)) \cdot 5{,}0$$

`backend/algorithms/exploration.py | FrontierExplorer.select_best_frontier`

Bonus additivo che spinge lo Scout verso le zone meno esplorate, complementare allo scaling multiplicativo di S_{eff}.

Bonus Momento.

$$B_{\text{mom}}(f) = \max(0,\ \cos\theta) \cdot 3{,}0$$

`backend/algorithms/exploration.py | FrontierExplorer.select_best_frontier`

dove θ è l'angolo tra la direzione corrente dello Scout (dal target attuale) e la direzione verso la frontiera f. Questo bonus impedisce che lo Scout cambi continuamente direzione quando compare una nuova frontiera laterale leggermente più appetibile: lo forza a continuare nella direzione già intrapresa.

2.2.2 Filtro Zone Esplorate (Solo Unknown Mode)

Prima della valutazione dell'utilità, in modalità unknown le frontiere vengono pre-filtrate in base alla copertura locale: se $r_{\text{exp}}(f) > 0{,}85$, la frontiera viene scartata perché si trova in una zona quasi completamente esplorata e spostarsi per coprire poche celle residue non è efficiente. Questo filtro non si applica in `map_known`, dove il routing a zone (Sezione 2.3) gestisce la stessa problematica in modo più strutturato.

2.2.3 Blocco del Target (Target Lock)

Quando una frontiera viene selezionata, lo Scout imposta un *target lock* per un numero configurabile di passi (`_TARGET_LOCK_DURATION`). Durante questo periodo, la rivalutazione completa delle frontiere è sospesa anche se ne compaiono di nuove. Il lock viene rilasciato anticipatamente se il target è raggiunto, se il rilevatore di stallo si attiva, o se la durata scade. Questo meccanismo previene cambi di direzione erratici quando lo Scout entra in zone con molti cluster di frontiera adiacenti.

2.2.4 Blacklist dei Target Recenti

Dopo aver raggiunto una frontiera, la sua posizione viene inserita in una blacklist per `_RECENT_TARGET_TTL` passi (default 200). Ciò previene l'oscillazione immediata tra cluster appena esplorati.

2.3 Routing a Zone (Map_Known)

In modalità `map_known`, tutte le frontiere della griglia sono visibili globalmente e la funzione di utilità classica non basta a impedire che lo Scout "rosicchi" bordi residui di zone già quasi completamente esplorate. Per risolvere questo problema, lo Scout suddivide la griglia in **zone macro** e filtra le frontiere candidate alla sola zona target, concentrando l'esplorazione in un blocco alla volta.

2.3.1 Suddivisione della Griglia

Il parametro `zone_divisions` (default 6) specifica il *numero totale* di zone. Il metodo `_zone_grid()` fattorizza questo numero in (c, r) colonne e righe scegliendo la combinazione che rende ogni zona il più possibile quadrata:

$$(\text{cols}, \text{rows}) = \arg\min_{\substack{c \cdot r = n \\ c \mid n}} \left| \frac{W}{c} - \frac{H}{r} \right|$$

`backend/agents/scout_agent.py | ScoutAgent._zone_grid`

Ad esempio, su una griglia 25×25 con $n = 6$, il risultato è 2×3 (zone da $12{,}5 \times 8{,}3$ celle), anziché 6×1 o 1×6.

2.3.2 Selezione della Zona Target

Il metodo `_select_target_zone()` assegna un punteggio a ciascuna zona combinando la frazione inesplorata (dominante) con una piccola componente di distanza:

$$\text{score}(z) = r_{\text{unk}}(z) - \frac{d(z)}{W + H} \cdot 0{,}15$$

`backend/agents/scout_agent.py | ScoutAgent._select_target_zone`

dove $r_{\text{unk}}(z)$ è la frazione di celle con `vision_explored == 0` nella zona e $d(z)$ è la distanza Manhattan dal centroide della zona. Lo Scout seleziona la zona con punteggio massimo.

Due meccanismi prevengono il flip-flop tra zone:

- **Anti-flip-flop**: Lo Scout deve restare nella zona corrente per almeno $2 \times$ `_TARGET_LOCK_DURATION` passi prima di poter cambiare, a meno che la zona non sia già completamente esplorata ($r_{\text{unk}} < 5\%$).
- **Isteresi**: Se la zona corrente ha ancora più del 15% di inesplorato *e* il suo rapporto è entro il 10% della zona migliore, lo Scout resta nella zona corrente anziché spostarsi.

2.3.3 Filtro Hard delle Frontiere

Dopo la selezione della zona target, le frontiere vengono filtrate spazialmente: solo quelle che cadono entro i limiti (x_0, y_0, x_1, y_1) della zona selezionata sono considerate. Se nessuna frontiera cade nella zona (zona completamente esplorata), viene calcolato un *waypoint* come centroide delle celle ancora inesplorate nella zona, utilizzato come bonus di momento per guidare lo Scout verso di essa.

Questo filtro non si applica in modalità unknown, dove il routing naturale delle frontiere combinato con il filtro $r_{\text{exp}}(f) > 0{,}85$ è sufficiente.

2.4 Anti-Clustering

L'anti-clustering è implementato in due livelli. Il primo è un filtro hard: le frontiere che si trovano a meno di `_ANTI_CLUSTER_DIST` celle (default 8) da un altro Scout vengono escluse. Il secondo è la penalizzazione soft nella funzione di utilità (P_{locale} e P_{globale}). Il risultato pratico è una divisione organica del territorio senza coordinazione esplicita.

2.5 Pattugliamento Ciclico (Stale Coverage)

Ogni Scout mantiene una matrice privata `_coverage_step` che registra il passo del modello in cui ciascuna cella è stata l'ultima volta nel raggio di visione.

Quando non esistono frontiere inesplorate, lo Scout entra in modalità di *stale coverage patrol*: le celle non visitate per un numero configurabile di passi (`_RESCAN_AGE`) diventano candidati per la ri-esplorazione. La selezione combina età e prossimità:

$$\text{score}(c) = \frac{\text{age}(c)}{d(c) + 1}$$

In modalità `map_known`, le celle con `vision_explored == 0` hanno priorità assoluta rispetto al ri-pattugliamento di celle già scansionate.

Capitolo 3

Coordinator Agent

Il Coordinator è l'unico agente con visione globale *aggregata*: non può vedere tutta la griglia, ma riceve informazioni da tutti gli Scout vicini e costruisce un modello del mondo più completo rispetto ai Retriever. Il suo ruolo è pianificare chi deve recuperare cosa, evitando ridondanze e massimizzando l'efficienza del sistema.

3.1 Raccolta delle Informazioni

Ogni volta che riceve un `ObjectSpottedMessage` da uno Scout, il Coordinator aggiorna il proprio `known_objects` con la posizione e il valore dell'oggetto. Ogni `TaskStatusMessage` ricevuto da un Retriever aggiorna uno dei dizionari interni: `retriever_carrying` (numero di oggetti trasportati), `retriever_energy` (energia residua) e `objects_being_collected` (set degli oggetti già in recupero).

Il Coordinator estrae la coda di task dichiarata esplicitamente da ciascun Retriever nel `TaskStatusMessage`. Gli slot disponibili per nuove assegnazioni vengono calcolati come:

$$\text{available_slots}(r) = \text{capacity}(r) - |\text{task_queue}(r)| - \text{carrying}(r)$$

`backend/agents/coordinator_agent.py | CoordinatorAgent._plan_task_assignments`

Questo approccio, basato sulla dichiarazione autoritativa del Retriever anziché su stime indirette, elimina le race condition tra assegnazione e auto-assegnazione.

3.2 Logica di Assegnazione

La funzione `_plan_task_assignments` calcola per ogni coppia (Retriever, oggetto) una priorità greedy:

$$\text{priority}(o, r) = \frac{\text{value}(o)}{d(r, o) + 1} + b_{\text{spotted}}$$

`backend/agents/coordinator_agent.py | CoordinatorAgent._plan_task_assignments`

dove $b_{\text{spotted}} = 0{,}5$ se il Retriever ha già visto l'oggetto direttamente (campo `_spotted_by`), altrimenti 0. I candidati vengono ordinati per priorità decrescente e assegnati in modo greedy rispettando i slot liberi di ciascun Retriever. Al momento dell'invio, il Coordinator trasmette sia il `TaskAssignmentMessage` che un `MapDataMessage` con le celle magazzino note, accelerando la scoperta delle stazioni di deposito da parte dei Retriever che non le hanno ancora esplorate.

Il set `objects_being_collected` è la protezione principale contro la doppia assegnazione: qualsiasi oggetto già assegnato o dichiarato in recupero da un Retriever viene escluso dalla lista di candidati per le nuove assegnazioni.

3.3 Strategia di Movimento

3.3.1 Riposizionamento verso il Centroide

Quando il Coordinator non ha compiti da assegnare, il suo obiettivo di movimento è rimanere raggiungibile dalla propria flotta. All'avvio, finché non riceve nessuna posizione recente da un Retriever, il Coordinator percorre un circuito di cinque waypoint strategici (centro e quattro quadranti della griglia, agganciati alla cella libera più vicina) per coprire tutta la griglia e ristabilire il contatto nel minor numero di passi.

Non appena arrivano posizioni fresche (più recenti di `_POS_MAX_AGE` passi, default 25), calcola il centroide medio di tutti i Retriever e, se la propria distanza da quel punto supera una soglia (pari a $\frac{1}{4}$ del raggio di comunicazione), si sposta verso la cella libera più vicina al centroide.

Il centroide può essere opzionalmente biased verso le posizioni degli oggetti pendenti tramite il parametro `_OBJECT_BIASED_CENTROID`:

$$\text{centroid}_{\text{biased}} = \text{centroid}_{\text{ret}} \cdot (1 - b) + \text{centroid}_{\text{obj}} \cdot b$$

`backend/agents/coordinator_agent.py | CoordinatorAgent._decide_exploration`

dove $b = 0{,}05$ (default). Questo attira leggermente il Coordinator verso le zone ad alta densità di oggetti senza allontanarlo dai Retriever.

3.3.2 Pattugliamento Anti-Inerzia (Boredom Patrol)

Se il Coordinator rimane inattivo per più di `_BOREDOM_THRESHOLD` passi (default 50) senza ricevere nuovi target di esplorazione, forza un ciclo di pattugliamento attraverso i waypoint strategici. Questo impedisce l'accumulo di energia sprecata in attesa passiva quando i task sono pochi e il centroide è già raggiunto.

3.3.3 Ricerca Attiva dei Retriever (Seek-Retriever)

Se esistono oggetti noti ma nessun Retriever è nel raggio di comunicazione, il Coordinator entra in modalità **SEEK-RETRIEVER**: si dirige verso l'ultima posizione dichiarata di un Retriever noto, con l'obiettivo di ristabilire il canale di comunicazione e procedere con l'assegnazione appena possibile. Solo le posizioni più recenti di `_POS_MAX_AGE` passi vengono considerate; le posizioni stale vengono ignorate per evitare di inseguire fantasmi. Se tutte le posizioni note sono stale o già visitate senza successo, il Coordinator torna al pattugliamento.

3.4 Consapevolezza Spaziale

3.4.1 Rilevamento Chokepoint

Il Coordinator valuta se la propria posizione corrente è un *chokepoint*: una cella con al massimo 2 vicini walkable. In tal caso, si sposta per evitare di bloccare il passaggio dei Retriever in corridoi stretti.

3.5 Esplorazione in Modalità map_known

In modalità `map_known`, il Coordinator usa `vision_explored == 0` come maschera di inesplorato. Quando é già vicino al centroide, tende verso le celle ignote più prossime ad esso; in assenza di task le tratta come frontiere normali. Le celle boundary con oltre l'85% della finestra 7×7 già esplorate vengono filtrate per evitare micro-frange residue.

3.6 Sincronizzazione tra Coordinator Multipli

In configurazioni con più di un Coordinator, il rischio di doppia assegnazione aumenta. Il `CoordinatorSyncMessage` viene scambiato periodicamente tra Coordinator vicini e contiene la lista delle assegnazioni attive (`assigned_tasks`) e degli oggetti in recupero (`objects_being_collected`). Alla ricezione, ogni Coordinator fonde le informazioni ricevute con le proprie, aggiornando i set interni.

La sincronizzazione è soggetta a rate limiting: un solo messaggio ogni `_SYNC_RATE_LIMIT` passi (default 10) per coppia di Coordinator, controllato tramite timestamp per-Coordinator nel dizionario `last_sync_step`. Questa propagazione non è istantanea, si propaga a onde tramite la comunicazione locale, ma è sufficiente per eliminare la stragrande maggioranza delle doppie assegnazioni in scenari realistici, bilanciando correttezza e overhead comunicativo.

Capitolo 4

Retriever Agent

Il Retriever è l'agente più complesso: deve recuperare oggetti fisicamente sulla griglia, gestire la capacità di carico, navigare verso il magazzino, ricaricarsi e uscire, tutto mantenendo una comunicazione attiva con Coordinator e altri Retriever.

4.1 Gerarchia delle Priorità

Il ciclo decisionale del Retriever è una gerarchia di priorità decrescenti, valutate in sequenza ad ogni passo della fase `STAGE_DECIDE`:

1. **P1 — Consegna**: Se trasporta almeno un oggetto e la capacità è esaurita (oppure la coda è vuota), si dirige al magazzino tramite la sequenza di consegna. In caso di capacità piena, tutti i claim residui nella coda vengono rilasciati immediatamente, rendendoli disponibili per i peer.

2. **P2 — Ricarica d'emergenza**: Se l'energia scende sotto la soglia critica e non sta già consegnando, il Retriever interrompe qualsiasi operazione e si dirige al magazzino per ricaricarsi. La soglia è dinamica: se il magazzino più vicino noto dista d celle, la soglia di allarme è $d \times 1.5$ unità di energia, garantendo sempre un margine di sicurezza.

3. **P3 — Esecuzione compiti assegnati**: Se ha una coda di compiti assegnati dal Coordinator, preleva il prossimo oggetto dalla coda e si dirige verso di esso. La coda viene riordinata per distanza Manhattan dalla posizione corrente ad ogni passo (`_TASK_QUEUE_REORDER`), garantendo che il task più vicino sia sempre il primo. Un task viene invalidato e rimosso se la sua posizione non è presente né in `known_objects` né in `dubious_objects`.

4. **P3b — Pickup opportunistico**: Dopo aver selezionato il task primario, il Retriever scansiona `known_objects` alla ricerca di oggetti non reclamati entro il raggio di visione. Se la capacità residua lo consente, reclama atomicamente fino a `spare_capacity` oggetti aggiuntivi. Questo massimizza il fattore di utilizzo del carico durante il transito verso il task primario.

5. **P4 — Auto-assegnazione (modalità hive-mind)**: In assenza di compiti assegnati, il Retriever scansiona l'intero `known_objects` accumulato (solo oggetti confermati dalla propria visione diretta) e si auto-assegna l'oggetto più conveniente tramite `try_claim_object` atomico. Questo meccanismo elimina i tempi morti tra un'assegnazione e l'altra.

6. **P4b — Verifica oggetti dubbi**: Se P4 non produce candidati, il Retriever consulta `dubious_objects` (Sezione 4.8) e si dirige verso l'oggetto dubbio più vicino per verificarne la presenza. All'arrivo, se l'oggetto esiste viene confermato in `known_objects`; altrimenti viene creato un tombstone. Questa verifica avviene solo a mani vuote.

7. **P5 — Mini-esplorazione**: Se non ha né compiti né oggetti da verificare, il Retriever esegue una frontier-based exploration per scoprire oggetti nell'area circostante (Sezione 4.9).

4.2 Auto-Assegnazione con Quattro Livelli di Sicurezza

La funzione `_try_self_assign_visible` è protetta da quattro filtri in cascata per prevenire doppie assegnazioni:

1. **Verifica sulla griglia**: Se l'oggetto non è più presente in `model.grid.objects`, viene scartato.

2. **Età del claim globale**: Se un altro agente ha un claim fresco (meno di `_STALE_CLAIM_AGE` passi, default 50) sull'oggetto, viene saltato.

3. **Controllo code peer**: Se l'oggetto appare nella `task_queue` di un Retriever nelle vicinanze, viene saltato.

4. **Claim atomico**: `CommunicationManager.try_claim_object` garantisce first-come-first-served a livello globale.

I claim più vecchi di `_STALE_CLAIM_AGE` passi sono considerati scaduti e possono essere acquisiti da un altro Retriever. Questo meccanismo di *stale claim takeover* previene che un Retriever fuori energia o bloccato impedisca indefinitamente il recupero di un oggetto.

4.3 Cancellazione Reattiva dei Task

Alla ricezione di un `MapDataMessage`, il Retriever verifica immediatamente se gli oggetti nella propria coda sono ancora presenti sulla griglia. Se un oggetto risulta assente (già raccolto da un peer), il task viene rimosso e il claim rilasciato senza attendere la riassegnazione da parte del Coordinator. Questo riduce i passi sprecati verso oggetti fantasma.

4.4 Sequenza di Consegna al Magazzino

La navigazione al magazzino avviene in quattro sotto-fasi gestite da una macchina a stati interna (`_wh_step`):

1. **Approach**: Il Retriever si dirige verso la cella `WAREHOUSE_ENTRANCE` più vicina del magazzino ottimale (selezionato da `get_best_warehouse_for`). Il percorso A* usa le forbidden types per forzare il passaggio dall'ingresso, escludendo le celle di uscita come nodi intermedi.

2. **Deposit**: Entrato nel magazzino attraverso l'ingresso, il Retriever deposita tutti gli oggetti trasportati, che vengono conteggiati nelle metriche e rimossi dalla griglia. Lo stato passa a `RECHARGE` se l'energia è sotto soglia, altrimenti direttamente a `EXIT`.

3. **Recharge**: Il Retriever rimane sulla cella magazzino mentre viene ricaricato a tasso costante (`recharge_rate` unità per passo) fino al recupero completo dell'energia massima. Durante la ricarica continua a ricevere messaggi.

4. **Exit**: Il Retriever si dirige verso la cella `WAREHOUSE_EXIT` e la attraversa. Uscito dal magazzino, `_wh_step` viene azzerato; prima di tornare all'esplorazione, il Retriever tenta immediatamente una nuova auto-assegnazione tramite `known_objects`, riducendo i passi a vuoto tra una consegna e la successiva.

4.5 Selezione del Magazzino Ottimale

La funzione `get_best_warehouse_for` seleziona la stazione di deposito/ricarica ottimale tra tutte quelle note all'agente. Il punteggio per ogni stazione combina tre fattori:

- **Distanza effettiva A***: La distanza viene calcolata tramite il pathfinder A*, che tiene conto degli ostacoli reali sulla griglia. L'uso della distanza Manhattan causerebbe errori significativi in mappe con corridoi e barriere: un magazzino "vicino" in linea d'aria può richiedere un percorso effettivo doppio o triplo a causa degli ostacoli. Se il percorso non esiste, viene applicata una penalizzazione di +500 sulla distanza Manhattan come fallback.

- **Congestione**: Il numero di Retriever già diretti verso ciascun magazzino viene conteggiato esaminando `_wh_station` di tutti gli agenti. La penalizzazione è lineare: $\text{penalty} = 8 \times n_{\text{heading}}$.

- **Fattibilità energetica**: Se la distanza stimata (moltiplicata per un fattore di percorso 1.4 per tenere conto di deviazioni residue) supera l'energia residua dell'agente, la stazione riceve una penalizzazione aggiuntiva di 200 punti, rendendola di fatto non preferibile.

$$\text{score}(s) = d_{\text{A}*}(pos, s) + 8 \cdot n_{\text{heading}}(s) + \begin{cases} 200 & \text{se } d_{\text{A}*} \cdot 1{,}4 > E_{\text{agent}} \\ 0 & \text{altrimenti} \end{cases}$$

`backend/core/warehouse_model.py | WarehouseModel.get_best_warehouse_for`

4.5.1 Ri-routing per Congestione

Se il Retriever è in fase `approach` da almeno 6 passi e il numero di agenti diretti allo stesso ingresso supera la soglia `_WH_CONGESTION_THRESHOLD`, viene rivalutata la scelta del magazzino escludendo quello corrente. Se esiste un'alternativa con minor congestione, il Retriever cambia rotta immediatamente, azzerando il percorso A* e il contatore di approccio.

4.5.2 Ordinamento delle Celle Interne (Queueing)

Le celle interne al magazzino sono ordinate per distanza dall'ingresso in modo decrescente: la cella più lontana dall'ingresso è lo slot 0 (il più profondo). Gli agenti riempiono il magazzino dalla posizione più interna verso l'ingresso, garantendo che nessun agente si fermi direttamente sulla porta anche in condizioni di alta congestione.

4.6 Coordinamento tra Retriever (Peer Negotiation)

I conflitti tra Retriever che puntano allo stesso oggetto si risolvono in due livelli:

1. **Claim atomico**: Ogni auto-assegnazione passa per l'operazione atomica `try_claim_object` di `CommunicationManager`, condivisa da tutti gli agenti. Solo un Retriever ottiene il claim; i contendenti ricevono immediatamente un fallimento e selezionano un altro oggetto.

2. **Peer yield via TaskStatusMessage**: Dopo ogni claim riuscito, il Retriever diffonde un `TaskStatusMessage` a tutti i vicini. Alla ricezione, ogni peer rimuove dalla propria `task_queue` gli oggetti già presenti nella coda del mittente, rilasciando i claim corrispondenti e aggiornando preventivamente la propria pianificazione senza attendere un fallimento esplicito.

L'assenza di un arbitro centralizzato e la natura broadcast del `TaskStatusMessage` garantiscono convergenza rapida: nella pratica, le collisioni residue dopo il claim atomico sono rare e vengono risolte entro uno-due passi di comunicazione.

4.6.1 Broadcast Peer-to-Peer degli Oggetti Scoperti

I Retriever possono comunicare direttamente tra loro tramite `RetrieverEventMessage` con eventi `object_spotted` e `cargo_dropped`. Alla ricezione, la posizione dell'oggetto segnalato non viene inserita direttamente in `known_objects`, poiché il peer non ha visione diretta, ma in `dubious_objects` (Sezione 4.8). L'oggetto verrà verificato dal Retriever ricevente quando idle (P4b), oppure sarà promosso a `known_objects` non appena sarà avvistato direttamente. Questo crea catene di scoperta opportunistiche che propagano la conoscenza degli oggetti peer-to-peer senza intermediazione del Coordinator, contenendo al tempo stesso i falsi positivi.

4.6.2 Relay delle Posizioni

Prima di inviare un `TaskStatusMessage`, ogni Retriever aggiorna la propria posizione nel dizionario `retriever_positions`. Questa informazione viene inclusa nei `MapDataMessage` e propagata da tutti i riceventi, creando una rete di relay in cui ogni agente diventa un nodo di trasmissione per le posizioni di tutti i Retriever.

4.7 Tolleranza ai Guasti: Cargo Drop

Se l'energia del Retriever raggiunge zero, tutti gli oggetti trasportati vengono rilasciati sulle celle libere adiacenti in una spirale espansiva. Le posizioni degli oggetti rilasciati vengono trasmesse ai vicini tramite `RetrieverEventMessage` con evento `cargo_dropped`, permettendo ad altri Retriever di recuperarli. Questo meccanismo impedisce che un guasto energetico causi la perdita permanente degli oggetti già raccolti.

4.8 Sistema degli Oggetti Dubbi

Il Retriever mantiene due strutture parallele a `known_objects`:

- `dubious_objects`: dizionario posizione → valore dell'oggetto.
- `dubious_objects_step`: dizionario posizione → passo in cui l'informazione dubbia è stata registrata.

Una posizione viene classificata come *dubbia*, anziché confermata, in quattro circostanze:

1. **Relay MapDataMessage**: ogni oggetto che la classe base aggiunge a `known_objects` tramite il merge di un `MapDataMessage` viene immediatamente spostato in `dubious_objects`. Il Retriever non fiducia le informazioni di seconda mano sulla presenza di oggetti: solo la propria visione diretta costituisce conferma.
2. **Invecchiamento** (*stale-demote*): ogni oggetto in `known_objects` non re-confermato dalla visione diretta entro 30 passi viene demoted a dubious. Gli oggetti nella `task_queue` attiva sono esentati da questa politica.
3. **Segnalazione peer** (`RetrieverEventMessage`): oggetti comunicati da altri Retriever vanno in `dubious_objects` per default.
4. **TaskAssignmentMessage sconosciuto**: se il Coordinator assegna un target che il Retriever non ha mai visto, la posizione viene inserita in `dubious_objects` e poi accodata in `task_queue` per la verifica all'arrivo.

4.8.1 Potatura con Confronto Timestamp

Un tombstone in `known_objects_cleared` cancella una voce dubbia *solo se il tombstone è almeno altrettanto recente* della segnalazione dubbia stessa:

$$\texttt{known_objects_cleared.get}(p, -1) \geq \texttt{dubious_objects_step.get}(p, 0)$$

`backend/agents/retriever_agent.py | RetrieverAgent.step_sense`

Questa condizione impedisce che tombstone vecchi (registrati *prima* di ricevere un'informazione dubbia aggiornata) invalidino erroneamente la voce più nuova, evitando la perdita di leads validi. La potatura avviene in tre punti: durante `step_sense`, al termine di `process_received_messages`, e all'inizio della fase P4b.

4.9 Frontiere di Esplorazione

Quando il Retriever non ha né task né oggetti da verificare, richiama `_update_explore_target`, che a sua volta delega la scelta della frontiera a `FrontierExplorer.select_best_frontier`.

4.9.1 Persistenza del Momento alla Frontiera

Quando il Retriever raggiunge una frontiera, il suo valore viene salvato in `_last_explore_target` prima di azzerare il puntatore corrente. Il bonus momento usa questo riferimento se `_explore_target` è `None`, mantenendo la direzione già intrapresa e prevenendo il backtracking verso zone già coperte:

$$B_{\text{mom}}(f) = \max(0,\ \cos\theta) \cdot 3{,}0, \quad \theta = \angle(\overrightarrow{p_{\text{last}} \to p},\ \overrightarrow{p \to f})$$

`exploration.py | FrontierExplorer.select_best_frontier`

4.9.2 Esplorazione in Modalità `map_known`

In `map_known` la `local_map` è pre-caricata con la topologia reale: non esistono celle UNKNOWN, e dunque la ricerca classica di frontiere (bordi tra UNKNOWN e FREE) non trova nulla. L'algoritmo usa invece `vision_explored == 0` come maschera di inesplorato.

Rapporto di esplorazione obstacle-aware. In modalità `map_known`, la formula generica di r_{exp} non può usare `UNKNOWN` come maschera (la mappa è già interamente nota), né può dividere per tutte le celle della finestra, poiché le celle ostacolo sono pre-marcate come esplorate gonfiando artificialmente il rapporto vicino alle pareti. La versione obstacle-aware restringe sia il numeratore (celle visivamente scansionate) sia il denominatore alle sole celle walkable:

$$r_{\text{exp,mk}}(f) = \frac{\sum_{c \in W_f} \mathbf{1}[\texttt{vision_explored}(c) = 1]}{|W_f|}, \qquad W_f = \{c : \texttt{nav_map}(c) = 0\}$$

`RetrieverAgent._update_explore_target`

dove $\texttt{nav_map}(c) = 0$ indica celle percorribili (non ostacolo, non magazzino).

Stima della massa inesplorata. In modalità `map_known`, viene fornita a `select_best_frontier` una callback `unknown_mass_at` (in `_update_explore_target`) che conta le celle walkable e non ancora scansionate entro raggio 8 attorno a ciascuna frontiera:

$$M(f) = |\{c : \|c - f\|_\infty \leq 8, \quad \texttt{nav_map}(c) = 0,\ \texttt{vision_explored}(c) = 0\}|$$

`RetrieverAgent._update_explore_target`

Questo sostituisce la semplice dimensione del cluster di frontiera con una stima più fedele del territorio realmente da scandire, orientando il Retriever verso regioni con la maggiore densità di oggetti potenzialmente non ancora scoperti.

Capitolo 5

Benchmarks

I benchmark sono stati eseguiti su due configurazioni di magazzino (Preset A e Preset B) con lo stesso numero di run per garantire significatività statistica. Per ogni preset vengono riportati quattro grafici e una tabella riepilogativa.

5.1 Preset A

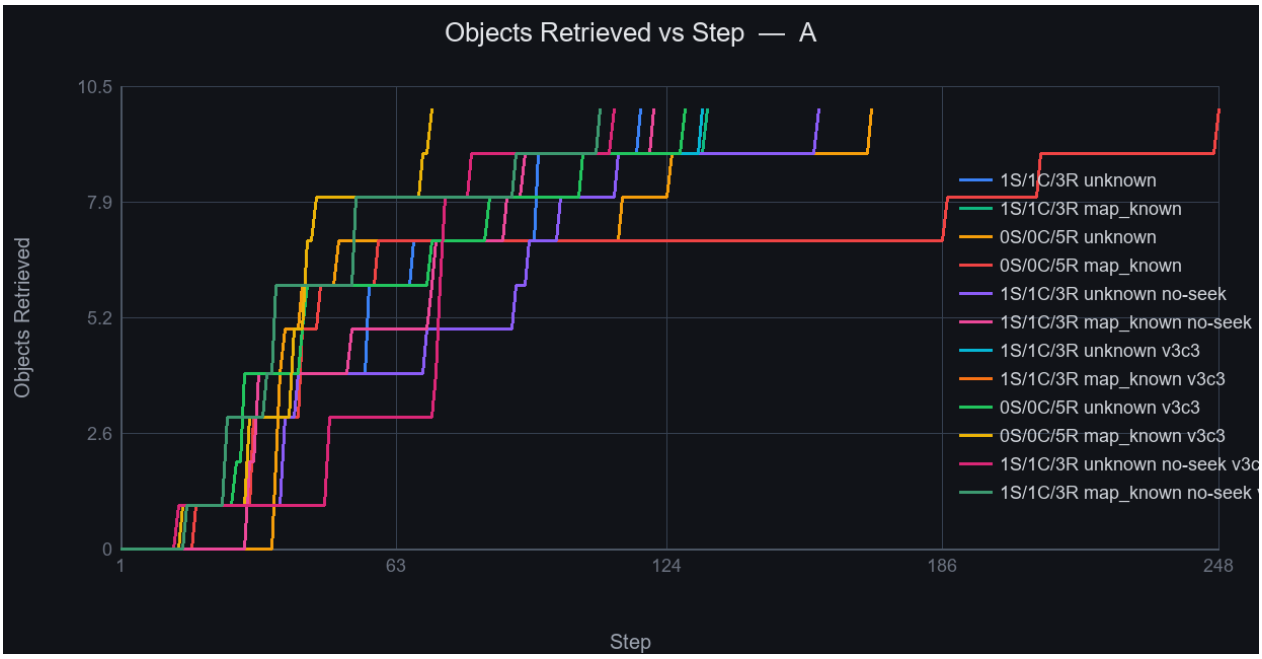

Figura 5.1: Andamento del recupero oggetti — Preset A

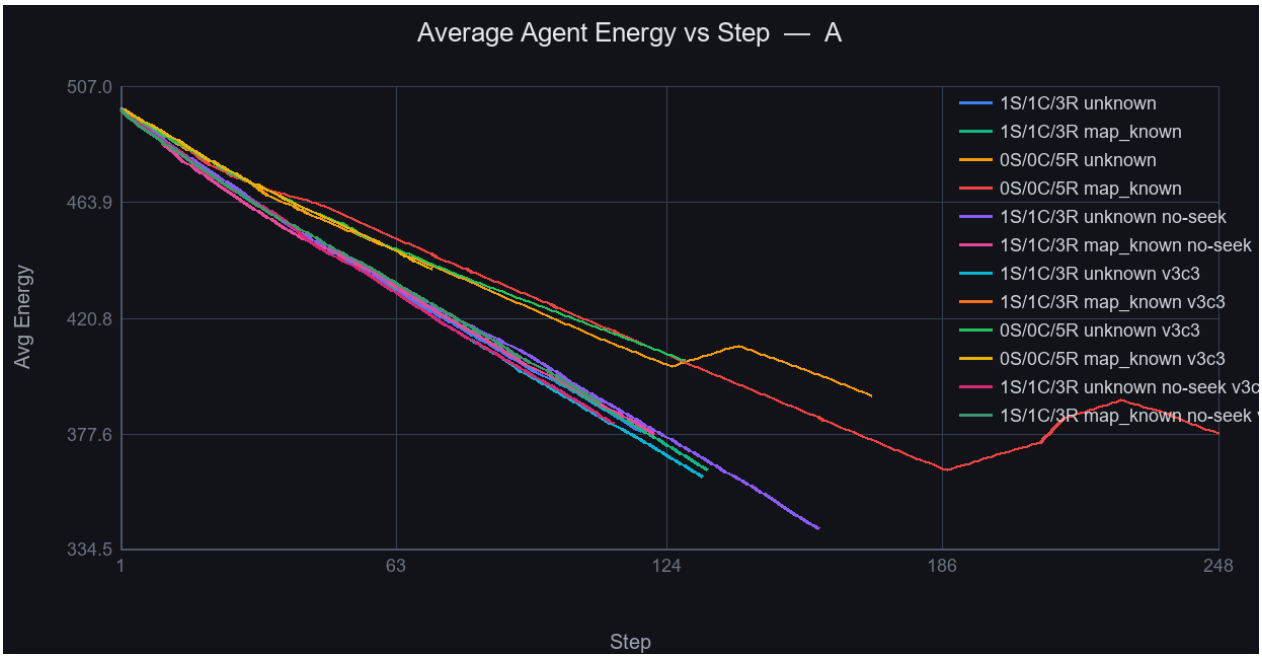

Figura 5.2: Consumo energetico — Preset A

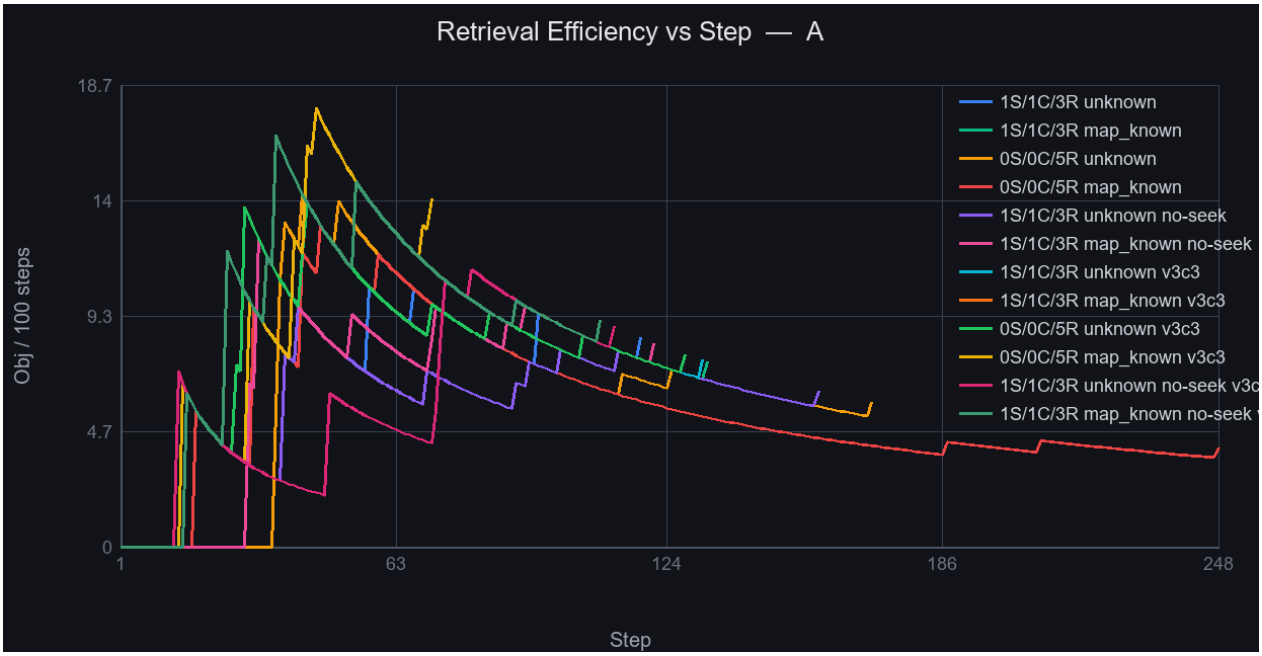

Figura 5.3: Efficienza energetica — Preset A

Figura 5.4: Messaggi scambiati — Preset A

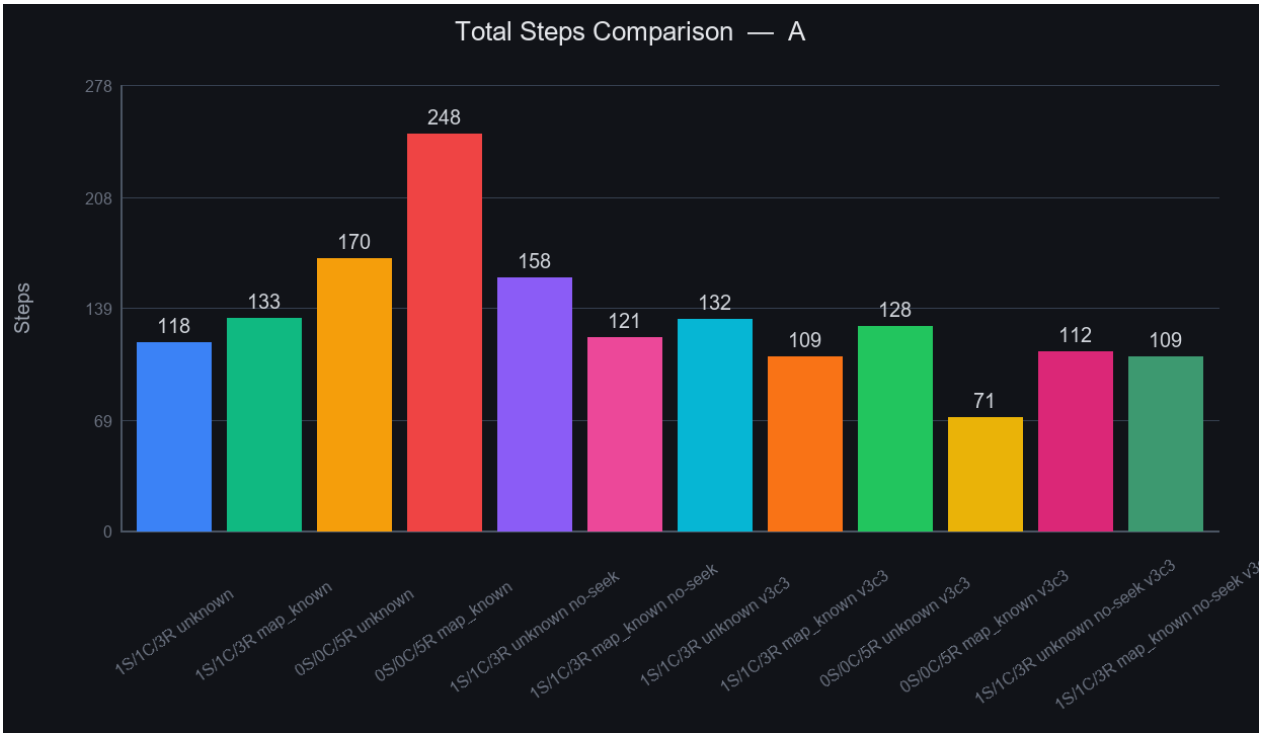

Figura 5.5: Confronto step totali — Preset A

Summary — A

Config	Steps	Retrieved	Completion	Efficiency	Avg Energy	Messages
1S/1C/3R unknown	118	10/10	100.0%	8.47	379	186
1S/1C/3R map_known	133	10/10	100.0%	7.52	364	251
0S/0C/5R unknown	170	10/10	100.0%	5.88	392	206
0S/0C/5R map_known	248	10/10	100.0%	4.03	378	416
1S/1C/3R unknown no-seek	158	10/10	100.0%	6.33	342	245
1S/1C/3R map_known no-seek	121	10/10	100.0%	8.26	378	244
1S/1C/3R unknown v3c3	132	10/10	100.0%	7.58	362	383
1S/1C/3R map_known v3c3	109	10/10	100.0%	9.17	389	255
0S/0C/5R unknown v3c3	128	10/10	100.0%	7.81	405	352
0S/0C/5R map_known v3c3	71	10/10	100.0%	14.08	439	226
1S/1C/3R unknown no-seek v3c3	112	10/10	100.0%	8.93	381	359
1S/1C/3R map_known no-seek v3c3	109	10/10	100.0%	9.17	389	255

Figura 5.6: Tabella riepilogativa — Preset A

5.1.1 Snapshot Finali — Preset A

(a) 0S-0C-5R — map_known

(b) 0S-0C-5R — unknown

(c) 0S-0C-5R — map_known, $v_r = 3, c_r = 3$

(d) 0S-0C-5R — unknown, $v_r = 3, c_r = 3$

Figura 5.7: Snapshot finali — 0S-0C-5R, Preset A

(a) 1S-1C-3R — map_known

(b) 1S-1C-3R — unknown

(c) 1S-1C-3R — map_known, $v_r = 3, c_r = 3$

(d) 1S-1C-3R — unknown, $v_r = 3, c_r = 3$

Figura 5.8: Snapshot finali — 1S-1C-3R, Preset A

(a) 1S-1C-3R — map_known, no-seek

(b) 1S-1C-3R — unknown, no-seek

(c) 1S-1C-3R — map_known, no-seek, $v_r = 3, c_r = 3$

(d) 1S-1C-3R — unknown, no-seek, $v_r = 3, c_r = 3$

Figura 5.9: Snapshot finali — 1S-1C-3R no-seek, Preset A

5.2 Preset B

Figura 5.10: Andamento del recupero oggetti — Preset B

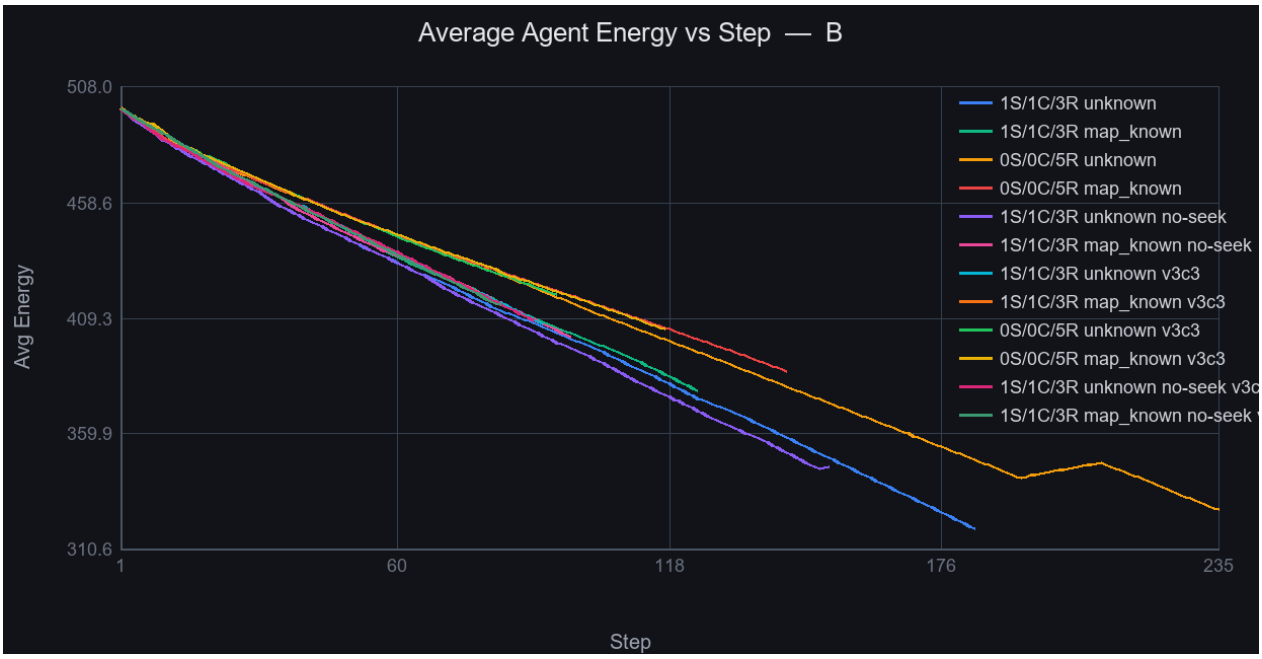

Figura 5.11: Consumo energetico — Preset B

Figura 5.12: Efficienza energetica — Preset B

Figura 5.13: Messaggi scambiati — Preset B

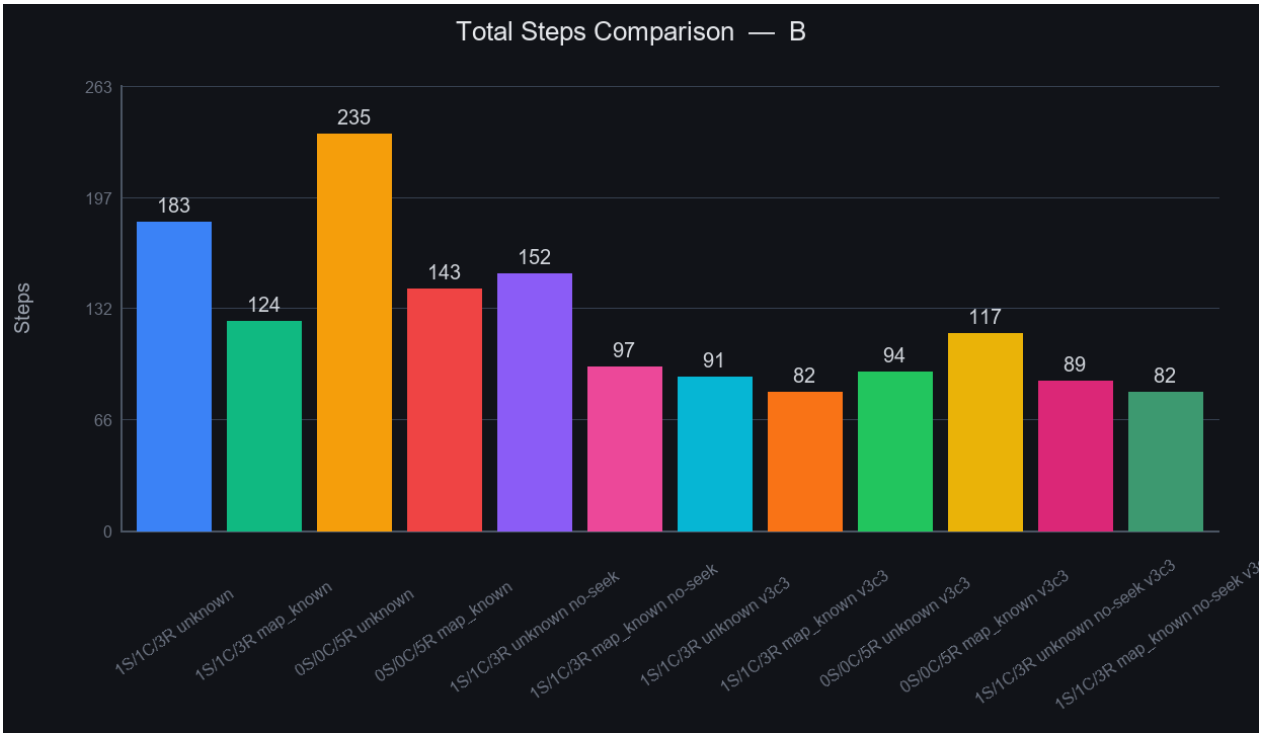

Figura 5.14: Confronto step totali — Preset B

Summary — B

Config	Steps	Retrieved	Completion	Efficiency	Avg Energy	Messages
1S/1C/3R unknown	183	10/10	100.0%	5.46	320	311
1S/1C/3R map_known	124	10/10	100.0%	8.06	378	297
0S/0C/5R unknown	235	10/10	100.0%	4.26	328	359
0S/0C/5R map_known	143	10/10	100.0%	6.99	387	208
1S/1C/3R unknown no-seek	152	10/10	100.0%	6.58	346	346
1S/1C/3R map_known no-seek	97	10/10	100.0%	10.31	401	264
1S/1C/3R unknown v3c3	91	10/10	100.0%	10.99	407	322
1S/1C/3R map_known v3c3	82	10/10	100.0%	12.20	415	332
0S/0C/5R unknown v3c3	94	10/10	100.0%	10.64	419	332
0S/0C/5R map_known v3c3	117	10/10	100.0%	8.55	405	439
1S/1C/3R unknown no-seek v3c3	89	10/10	100.0%	11.24	408	306
1S/1C/3R map_known no-seek v3c3	82	10/10	100.0%	12.20	415	332

Figura 5.15: Tabella riepilogativa — Preset B

5.2.1 Snapshot Finali — Preset B

(a) 0S-0C-5R — map_known

(b) 0S-0C-5R — unknown

(c) 0S-0C-5R — map_known, $v_r = 3, c_r = 3$

(d) 0S-0C-5R — unknown, $v_r = 3, c_r = 3$

Figura 5.16: Snapshot finali — 0S-0C-5R, Preset B

(a) 1S-1C-3R — map_known

(b) 1S-1C-3R — unknown

(c) 1S-1C-3R — map_known, $v_r = 3, c_r = 3$

(d) 1S-1C-3R — unknown, $v_r = 3, c_r = 3$

Figura 5.17: Snapshot finali — 1S-1C-3R, Preset B

(a) 1S-1C-3R — map_known, no-seek

(b) 1S-1C-3R — unknown, no-seek

(c) 1S-1C-3R — map_known, no-seek, $v_r = 3, c_r = 3$

(d) 1S-1C-3R — unknown, no-seek, $v_r = 3, c_r = 3$

Figura 5.18: Snapshot finali — 1S-1C-3R no-seek, Preset B